

SeuGado — João: o plano semanal

Otimizador guloso semanal: do estado do pasto à recomendação de manejo

27/09/2026

Sua parte em uma frase: o cérebro da recomendação. Você recebe a fotografia da fazenda (quanto capim cada piquete tem hoje, quanto vai crescer em cada um dos próximos dias e quanto cada lote come) e devolve o **plano da semana**: quais lotes mudar de piquete, em que dia, para onde, por quê e com que confiança.

Você entrega: `planner/otimizador.py`, `planner/confianca.py`, `planner/geo.py` e `persistencia/planos.py`, com testes. Seu código não tem tela nem rede; é Python puro, mais duas funções de banco.

Prazo: PR aberto até domingo à noite.

1. O SeuGado em um minuto

O gado é dividido em **lotes**, e o pasto em **piquetes**. O capim tem um ponto certo para o lote **entrar** (altura de entrada) e um ponto em que o lote precisa **sair** (altura de saída). Se o gado entra tarde, o capim já passou do ponto e perdeu valor. Se sai tarde, a planta fica rapada demais e o pasto se degrada por temporadas. **Esse é o erro mais caro.**

O Kauê estima o capim de cada piquete por satélite e clima. O Leo manda o plano pelo Telegram. **Você decide o plano.** Na call, a equipe combinou:

- o plano é **semanal** (7 dias a partir de hoje);
- movimentação **só nos dias de manejo preferidos** pelo produtor (ex.: segunda e quinta);
- a mão de obra é contada em **animais**: por dia, no máximo `funcionarios_disponiveis` × `animais_por_funcionario_dia` animais movidos (um lote nunca é dividido).

No MVP, só existe Marandu em pastejo rotacionado (ADR-025). A fixture de exemplo também tem um piquete contínuo e um de Mombaça; eles estão lá de propósito, para você testar que o algoritmo os ignora sem quebrar.

Isto é o **estágio 1** do motor de otimização do projeto (guloso, ADR-008). Busca local e CP-SAT vêm depois; o seu guloso vira a linha de base contra a qual eles serão medidos.

Palavras que você vai usar

Termo	O que é
Massa (kg MS/ha)	Quanto capim seco há por hectare. É o “estoque” do piquete
Altura (cm)	<code>massa ÷ densidade</code> . O produtor pensa em cm; toda comparação com alvo é em cm
Taxa de acúmulo (kg MS/ha/dia)	Quanto o capim cresce por dia. Vem pronta, uma por dia, para 14 dias

Termo	O que é
Consumo do lote (kg MS/dia)	Quanto o lote come por dia. Vem pronto
Eficiência de pastejo	Fração do capim removido que vira comida; o resto é pisoteado. Remoção do piquete por dia = $\text{consumo} \div \text{eficiência} \div \text{área}$
Descanso	Dias desde que o último lote saiu. O piquete só recebe lote depois de um descanso mínimo (21 dias no Marandu)
Apto	Piquete que pode receber lote: altura $\geq$ entrada e descanso cumprido e vazio
Urgência	$\text{altura_saida} - \text{altura_prevista}$. Quanto maior, mais urgente tirar o lote
Confiança	$\text{alta} / \text{media} / \text{baixa}$. A recomendação vale o que vale o dado mais fraco (ADR-019)
Rotacionado / contínuo	Só piquetes rotacionados entram no plano. No contínuo, o lote fica fixo, e você só gera alerta

Entradas e saídas: o que você recebe, de quem, e o que entrega

Esta seção é o seu contrato com o resto da equipe. **Nomes, tipos e formatos são exatamente estes**; se algo aqui parecer faltar ou estar errado, fale com o Kauê antes de inventar outro formato.

Resumo

	O quê	De quem / para quem	Como chega / sai
Recebe	EstadoProjetado (a fotografia da fazenda)	Kauê (montar_estado_projetado)	argumento estado de gerar_plano, passado pelo ciclo do Leandro
Recebe	Fazenda (rotina de manejo)	Leandro (carregar_fazenda)	argumento fazenda de gerar_plano
Recebe	agora: datetime (UTC, com fuso) e plano_id: UUID	Leandro (ciclo)	argumentos de gerar_plano
Recebe	funções prontas de cálculo	Kauê (core/, planner/estado.py)	import (lista abaixo)
Entrega	PlanoManejo	Leandro (salva e mostra na web), Leo (manda no Telegram), Ezequiel (pinta o mapa)	retorno de gerar_plano
Entrega	plano gravado (com status) + eventos manejo_recomendado	banco → Leo (respostas) e Leandro (página)	salvar_plano(conn, plano, status="vigente" "candidato") -> None
Entrega	troca de plano quando o produtor aceita o novo	Leo	promover_candidato(conn, plano_id) -> PlanoManejo, descartar_plano(conn, plano_id) -> None

	O quê	De quem / para quem	Como chega / sai
Entrega	o que mudou entre dois planos	Leandro (rotina diária) e Leo (mostra ao produtor)	<code>comparar_planos(anterior, novo, respondidas, hoje) -> tuple[DiferencaLote, ...]</code>
Entrega	movimentações já respondidas	Leandro e Leo	<code>ids_respondidos(conn, fazenda_id) -> frozenset[UUID]</code>
Entrega	leitura do plano atual	Leandro (rota /plano/atual) e Leo (bot)	<code>carregar_plano_atual(conn, fazenda_id) -> PlanoManejo None</code> e <code>carregar_plano(conn, plano_id) -> PlanoManejo None</code>

Assinaturas exatas que você implementa

```
# src/seugado/planner/otimizador.py
def gerar_plano(estado: EstadoProjetado, fazenda: Fazenda,
               agora: datetime, plano_id: UUID) -> PlanoManejo: ...

# src/seugado/planner/confianca.py
def confianca_movimentacao(destino: PiqueteProjetado,
                          lote: LoteProjetado) -> tuple[Confianca, str]: ...

# src/seugado/planner/geo.py
def distancia_m(lat1: float, lon1: float, lat2: float, lon2: float) -> float: ...

# src/seugado/planner/comparacao.py
def comparar_planos(anterior: PlanoManejo, novo: PlanoManejo,
                   respondidas: frozenset[UUID], hoje: date) -> tuple[DiferencaLote,
...]: ...

# src/seugado/persistencia/planos.py (nenhuma faz commit; conn: psycopg.Connection[Any])
def salvar_plano(conn, plano: PlanoManejo, status: str = "vigente") -> None: ...
def promover_candidato(conn, plano_id: UUID) -> PlanoManejo: ...
def descartar_plano(conn, plano_id: UUID) -> None: ...
def carregar_plano_atual(conn, fazenda_id: UUID) -> PlanoManejo | None: ... # vigente
def carregar_candidato(conn, fazenda_id: UUID) -> PlanoManejo | None: ...
def carregar_plano(conn, plano_id: UUID) -> PlanoManejo | None: ...
def status_do_plano(conn, plano_id: UUID) -> str | None: ...
def ids_respondidos(conn, fazenda_id: UUID) -> frozenset[UUID]: ...
```

Funções prontas que você usa (não reimplemente)

```
from seugado.planner.estado import avancar_massa_um_dia
# (massa_kg_ms_ha, taxa_acumulo_kg_ms_ha_dia, consumo_lote_kg_ms_dia, area_ha,
#  eficiencia_pastejo) -> float # estoque no início do dia seguinte
from seugado.core.regras import urgencia, descanso_cumprido, combinar_confianca
# urgencia(altura_atual_cm, parametros) -> float # saída - altura
# descanso_cumprido(dias_desde_ultima_saida: int, descanso_min_dias: float) -> bool
# combinar_confianca(*fatores: Confianca) -> Confianca # o mínimo
from seugado.core.forrageo import dias_ocupacao, massa_para_altura
# dias_ocupacao(massa_atual, massa_residuo, taxa, area_ha, eficiencia, consumo) -> float
(pode ser inf)
# massa_para_altura(massa_kg_ms_ha, densidade_kg_ha_por_cm) -> float
from seugado.contratos import (EstadoProjetado, PiqueteProjetado, LoteProjetado,
PlanoManejo,
    Movimentacao, Alerta, PedidoValidacao, ResumoPiquete, TipoAlerta,
    estado_de_dict, plano_para_dict, plano_de_dict)
from seugado.core.models import Fazenda, Confianca, MetodoPastejo
```

Formatos

EstadoProjetado: a fotografia da fazenda (entrada do otimizador)

Tipo Python em `seugado.contratos`. Exemplo completo:

`tests/fixtures/estado_projetado_exemplo.json` (carregue com `estado_de_dict`).

Campo	Tipo	Significado
<code>fazenda_id</code>	UUID	Fazenda
<code>data_base</code>	<code>date</code>	Hoje; primeiro dia do plano
<code>horizonte_previsao_dias</code>	<code>int</code>	14 (dias de taxa de crescimento prevista)
<code>piquetes</code>	<code>tuple[PiqueteProjetado, ...]</code>	Piquetes ativos, em ordem de nome
<code>lotes</code>	<code>tuple[LoteProjetado, ...]</code>	Lotes, em ordem de nome

PiqueteProjetado

Campo	Tipo	Unidade / valores	Significado
<code>piquete_id</code> , <code>nome</code>	UUID, <code>str</code>	—	Identificação
<code>area_ha</code>	<code>float</code>	ha	Área
<code>metodo_pastejo</code>	<code>MetodoPast</code> <code>ejo</code>	ROTACIONADO CONTINUO	Só rotacionado entra no plano
<code>cultivar_slug</code> , <code>cultivar_nome</code>	<code>str</code>	ex.: "marandu", "Marandu"	Capim
<code>centroide_lat</code> , <code>centroide_lon</code>	<code>float</code>	graus	Centro do piquete (para a distância)
<code>situacao</code>	<code>SituacaoPi</code> <code>quete</code>	OCUPADO DESCANSANDO	Situação hoje
<code>lote_atual_id</code>	UUID None	—	Lote que está nele hoje
<code>dias_descanso</code>	<code>int</code>	dias	Dias desde que o último lote saiu (0 se ocupado)
<code>parametros</code>	<code>Parametros</code> <code>Regime</code> None	—	Alturas-alvo: <code>altura_entrada_cm</code> , <code>altura_saida_cm</code> (rotacionado) ou <code>altura_maxima_cm</code> , <code>altura_minima_cm</code> (contínuo); mais <code>confianca</code> e <code>fonte</code>
<code>faltantes</code>	<code>tuple[str, ...]</code>	—	Vazio = pode planejar. Se não vazio, o piquete não recebe nem perde lote
<code>descanso_min_dias</code>	<code>float</code>	dias	Descanso mínimo do capim (21 no Marandu)
<code>densidade_kg_ha_por_cm</code>	<code>float</code> None	kg MS/ha por cm	Converte massa ↔ altura
<code>eficiencia_pastejo</code>	<code>float</code> None	0-1	Fração do capim removido que vira comida

Campo	Tipo	Unidade / valores	Significado
massa_hoje_kg_ms_ha	float None	kg MS/ha	Estoque hoje (None = sem estimativa)
altura_hoje_cm	float None	cm	massa ÷ densidade
taxa_acumulo_prevista_kg_ms_ha_dia	tuple[float, ...]	kg MS/ha/dia	14 valores: índice 0 = hoje, 1 = amanhã...
confianca, motivo_confianca	Confianca, str	—	Confiança da estimativa do piquete
dias_desde_image_limpa	int None	dias	Idade da última imagem de satélite sem nuvem

LoteProjetado

Campo	Tipo	Unidade / valores	Significado
lote_id, nome	UUID, str	—	Identificação
composicao	tuple[ComposicaoLote, ...]	cada item: categoria, n_animais, peso_medio_kg, origem_peso	Animais do lote
indissoluvel	bool	—	Nunca pode ser juntado a outro
piquete_atual_id	UUID None	—	Onde está hoje
desde	date None	—	Desde quando está lá
peso_vivo_total_kg	float	kg	Soma dos pesos
consumo_kg_ms_dia	float	kg MS/dia	Quanto o lote come por dia (já calculado)
confianca_peso, motivo_confianca_peso	Confianca, str	—	Alta se o peso foi informado; média se veio da tabela de UA

Fazenda: a configuração (de seu_gado.core.models)

Campo	Tipo	Valores	Significado
id	UUID	—	Fazenda
nome	str	—	Nome
timezone	str	"America/Fortaleza"	Fuso
funcionarios_disponiveis	int	≥ 1	Pessoas que fazem o manejo
animais_por_funcionario_dia	int	≥ 1	Quantos animais cada pessoa consegue mover por dia
dias_preferenciais_manejo	tuple[int, ...]	0 = segunda ... 6 = domingo	Dias em que se pode mover gado

Campo	Tipo	Valores	Significado
envio_plano_dia	int	0 = segunda ... 6 = domingo	Dia em que o produtor recebe o plano semanal
envio_plano_hora	int	0 ... 23 (hora local)	Hora em que recebe o plano
ativo	bool	—	Fazenda ativa

PlanoManejo: o plano da semana

Tipo Python em `seugado.contratos`; em JSON via `plano_para_dict` / `plano_de_dict`; em TypeScript em `frontend/src/lib/tipos.ts`. Exemplo completo: `tests/fixtures/plano_exemplo.json`.

Campo	Tipo (Python)	Em JSON	Significado
id	UUID	texto	Id do plano
fazenda_id	UUID	texto	Fazenda dona do plano
data_geracao	datetime (UTC, com fuso)	"2026-09-28T08:00:00+00:00"	Quando foi gerado
data_inicio	date	"2026-09-28"	Primeiro dia do plano (hoje)
horizonte_dias	int	7	Sempre 7
movimentacoes	tuple[Movimentacao, ...]	lista	Ordenadas por (data, lote_nome)
alertas	tuple[Alerta, ...]	lista	Ordenados por (data, tipo, texto)
pedidos_validacao	tuple[PedidoValidacao, ...]	lista	Ordenados pelo nome do piquete
piquetes	tuple[ResumoPiquete, ...]	lista	Um por piquete ativo, em ordem de nome

Movimentacao: “mover o lote X do piquete A para o B no dia D”

Campo	Tipo	Em JSON	Significado
id	UUID	texto	Id da movimentação; é o <code>entidade_id</code> do evento <code>manejo_recomendado</code> e o que vai nos botões do bot
data	date	"2026-09-28"	Dia da movimentação (sempre dia de manejo preferido)
lote_id / lote_nome	UUID / str	texto	Lote que muda
piquete_origem_id / piquete_origem_nome	UUID None / str None	texto ou null	De onde sai
piquete_destino_id / piquete_destino_nome	UUID / str	texto	Para onde vai
altura_destino_cm	float (1 casa)	número	Altura prevista do destino no início do dia
altura_entrada_alvo_cm	float	número	Altura ideal de entrada do capim do destino

Campo	Tipo	Em JSON	Significado
altura_origem_cm	float None	número ou null	Altura prevista da origem no início do dia
altura_saida_alvo_cm	float None	número ou null	Altura de saída do capim da origem
dias_previstos	int	número	Dias que o lote deve ficar no destino
motivo	str	texto	Frase pronta, em português, para o produtor
confianca	Confianca	"alta" "media" "baixa"	Confiança da recomendação
motivo_confianca	str	texto	Frase pronta: o dado mais fraco por trás da recomendação

Alerta

Campo	Tipo	Em JSON	Significado
tipo	TipoAlerta	um de: sem_piquete_apto, capacidade_excedida, aguardando_parametro, estimativa_indisponivel, continuo_acima_maxima, continuo_abaixo_minima, lote_sem_piquete, sem_dia_de_manejo, passando_do_ponto	Tipo do aviso
data	date	"2026-10-01"	Dia a que se refere
texto	str	texto	Frase pronta para o produtor
confianca / motivo_confianca	Confianca / str	texto	Como na movimentação
piquete_id / lote_id	UUID None	texto ou null	A quem se refere

PedidoValidacao: pedido de medir o capim com régua

Campo	Tipo	Em JSON	Significado
piquete_id / piquete_nome	UUID / str	texto	Piquete a medir
motivo	str	texto	Frase pronta: por que o sistema está pedindo

ResumoPiquete: a fotografia de cada piquete (usada no mapa)

Campo	Tipo	Em JSON	Significado
piquete_id / nome	UUID / str	texto	Piquete
situacao	SituacaoPiquete	"ocupado" "descansando"	Tem gado ou está descansando
lote_atual_nome	str None	texto ou null	Lote que está nele
altura_hoje_cm	float None	número ou null	Altura estimada hoje; null = sem estimativa

Campo	Tipo	Em JSON	Significado
altura_entrada_alvo_cm / altura_saida_alvo_cm	float None	número ou null	Alvos (nulos no contínuo ou sem parâmetro)
confianca / motivo_confianca	Confianca / str	texto	Confiança da estimativa do piquete
faltantes	tuple[str, ...]	lista de texto	O que falta para estimar ou planejar; vazio = tudo certo

O que salvar_plano grava

1. Uma linha na tabela plano: (id, fazenda_id, gerado_em, data_inicio, horizonte_dias, payload, status), COM payload = plano_para_dict(plano) (JSON).
2. **Só para plano vigente:** um evento manejo_recomendado por movimentação (origem sistema, ator "planner", chave_idempotencia = "recomendacao:{plano.id}:{mov.id}"), com o payload **exato**:

```
{
  "entidade_id": "<mov.id>",
  "lote_id": "<uuid>",
  "piquete_origem_id": "<uuid ou null>",
  "piquete_destino_id": "<uuid>",
  "data_prevista": "2026-09-28",
  "dias_previstos": 3,
  "motivo": "<texto>",
  "confianca": "media",
  "motivo_confianca": "<texto>"
}
```

2. Stack e onde fica o seu código

Python 3.12, sem dependência nova. Use o que já existe:

Onde	O que usar
seugado.contratos	Entrada: EstadoProjetado, PiqueteProjetado, LoteProjetado. Saída: PlanoManejo, Movimentacao, Alerta, PedidoValidacao, ResumoPiquete, TipoAlerta. Mais plano_para_dict/plano_de_dict e estado_de_dict
seugado.core.models	Fazenda (configuração), Confianca, MetodoPastejo
seugado.core.regras	urgencia, descanso_cumprido, combinar_confianca
seugado.core.forragem	massa_para_altura, dias_ocupacao
seugado.planner.estado	avancar_massa_um_dia (a física de um dia, a mesma que o Kauê usa)
seugado.persistencia.eventos	registrar_evento

Arquivos que são seus: src/seugado/planner/otimizador.py, src/seugado/planner/confianca.py, src/seugado/planner/geo.py, src/seugado/planner/comparacao.py, src/seugado/persistencia/planos.py, tests/planner/test_otimizador.py, tests/planner/test_comparacao.py, tests/planner/test_confianca.py, tests/persistencia/test_planos.py.

Não edite nenhum outro arquivo. Se precisar de algo em contratos.py ou core/, fale com o Kauê.

Regras de código do projeto (valem para você também)

- Type hints em toda função pública; nomes com unidade (`altura_cm`, `massa_kg_ms_ha`, `area_ha`).
- Termos do domínio em português (`piquete`, `lote`); docstrings e comentários em inglês.
- Nunca use entidade como chave de dicionário, só o `id`. Nunca compare membro de um enum com outro enum.
- Arquivo com mais de ~300 linhas → quebre em funções auxiliares em outro arquivo seu.
- Sem aleatoriedade: **mesma entrada → mesmo plano**.

3. Preparar o ambiente

Guia completo de instalação e comandos (Windows e Mac/Linux, token para o /docs, testes, Git e problemas comuns): `docs/equipe/COMO-RODAR.md`. Não rode migrações nem nada no Supabase: o banco já está pronto e é compartilhado.

Importante: o Python não lê o `.env` sozinho. Todo comando `uv run local` leva `--env-file .env` (como nos exemplos abaixo); sem isso a API responde erro 500 e os testes de banco são pulados.

1. Instale Git e **uv** (`docs.astral.sh/uv`). Clone o repositório e crie a sua branch: `feat/joao-otimizador`.
2. `uv sync --group dev`.
3. Rode `uv run --env-file .env pytest` para confirmar que tudo passa antes de você mexer.
4. Para `persistencia/planos.py` (tarefa J5), crie o `.env` a partir de `.env.example` com o `DATABASE_URL` e o `SEUGADO_TEST_DATABASE_URL` que o Kauê manda em privado.

Antes do PR: `uv run ruff check .`, `uv run mypy` e `uv run --env-file .env pytest` sem erro.

4. O contrato: o que entra e o que sai

```
def gerar_plano(
    estado: EstadoProjetado,
    fazenda: Fazenda,
    agora: datetime,          # timezone-aware (UTC); vira plano.data_geracao
    plano_id: UUID,          # quem chama gera (uuid4); nos testes, fixo
) -> PlanoManejo: ...
```

Entrada (`tests/fixtures/estado_projetado_exemplo.json` tem um exemplo completo; carregue com `estado_de_dict(json.load(...))`):

- `estado.data_base`: o primeiro dia do plano (hoje).
- Para cada piquete: `area_ha`, `metodo_pastejo`, `situacao`, `lote_atual_id`, `dias_descanso`, `parametros` (alturas de entrada e saída, ou máxima e mínima no contínuo), `faltantes`,

densidade_kg_ha_por_cm, eficiencia_pastejo, massa_hoje_kg_ms_ha, altura_hoje_cm, taxa_acumulo_prevista_kg_ms_ha_dia (**14 valores**: índice 0 = hoje), centroide_lat/lon, confianca, motivo_confianca.

- Para cada lote: piquete_atual_id, consumo_kg_ms_dia, indissoluvél, confianca_peso, motivo_confianca_peso.
- fazenda.dias_preferenciais_manejo: date.weekday() (0 = segunda ... 6 = domingo); funcionarios_disponiveis; animais_por_funcionario_dia.

Saída: PlanoManejo (exemplo completo em tests/fixtures/plano_exemplo.json). **Esse arquivo é exatamente o que o seu gerar_plano deve produzir para o estado de exemplo**, com a fazenda do exemplo (dias preferenciais segunda e quinta, 2 funcionários, 150 animais por funcionário por dia, envio segunda às 5h), agora = 2026-09-28T08:00:00+00:00 e plano_id = 44444444-4444-4444-8444-000000000001. Ele é o seu teste de aceitação principal.

5. Tarefas (na ordem)

J1: Distância (planner/geo.py)

Por quê: entre dois piquetes igualmente bons, o gado vai para o mais perto, o que evita rotas cruzadas pela fazenda (ADR-016). No MVP, a distância é em linha reta entre os centros dos piquetes.

Como: def distancia_m(lat1: float, lon1: float, lat2: float, lon2: float) -> float, pela fórmula de **Haversine** com raio da Terra de 6.371.000 m.

Critérios de aceite:

- [] Mesmo ponto → 0.
- [] 0,001° de latitude ≈ 111,2 m (± 0,5).

Fora do escopo: rota por porteira ou corredor; PostGIS.

J2: Confiança da movimentação (planner/confianca.py)

Por quê: toda recomendação diz o quanto dá para confiar nela e **por quê**, nomeando o elo mais fraco (ADR-019). É o que responde ao produtor quando ele pergunta “como eu sei que isso tá certo?”.

Como:

```
def confianca_movimentacao(destino: PiqueteProjetado, lote: LoteProjetado) -> tuple[Confianca, str]
```

1. Três fatores, **nesta ordem**:

- a. estimativa do piquete de destino: (destino.confianca, destino.motivo_confianca);
- b. parâmetro de altura: (destino.parametros.confianca, frase_parametro);
- c. peso do lote: (lote.confianca_peso, lote.motivo_confianca_peso).

2. `nivel = combinar_confianca(a, b, c)`. A frase é a do **primeiro** fator, na ordem acima, cujo nível é igual a `nivel`.
3. `frase_parametro`:
 - se `parametros.fonte` começa com "produtor:" → "a altura de entrada foi informada por você, não por fonte técnica";
 - senão → `f"a altura de entrada do {cultivar_nome} no pastejo {metodo} vem de fonte com confiança {nivel_do_parametro}"`, com `metodo` em rotacionado/contínuo e o nível em alta/média/baixa, com acento.

Critérios de aceite:

- [] Destino alta, parâmetro média, peso média → ("media", "a altura de entrada do Marandu no pastejo rotacionado vem de fonte com confiança média").
- [] Destino baixa → a frase do destino.

Fora do escopo: confiança numérica; multiplicar fatores.

J3: O otimizador guloso semanal (planner/otimizador.py)

Por quê: é a entrega central do SeuGado: transformar o estado do pasto numa ordem de manejo que respeita a agronomia e a rotina da fazenda (dias de manejo e mão de obra). Nenhum concorrente modela essa rotina.

Constantes:

```
HORIZONTE_DIAS = 7
FRACAO_ALVO_MARGINAL = 0.9      # HIPOTESE-CALIBRAR (ADR-024)
FATOR_PASSANDO_DO_PONTO = 1.2   # HIPOTESE-CALIBRAR (ADR-025)
```

Algoritmo (siga exatamente, porque a fixture depende disso):

Passo 0: preparar.

- `P` = piquetes por id; `L` = lotes por id.
- Um piquete é **planejável** se: `metodo_pastejo == ROTACIONADO`, `faltantes` vazio, `massa_hoje_kg_ms_ha` não nulo e `parametros` não nulo.
- Estado simulado, que você atualiza dia a dia: `massa[p]` (começa em `massa_hoje`), `ocupante[p]` (id do lote ou None, a partir de `lote.piquete_atual_id`), `posicao[l]` (o piquete do lote) e `descanso[p]` (começa em `dias_descanso`).
- `capacidade_animais = funcionarios_disponiveis × animais_por_funcionario_dia`.
- `n_animais(lote)` = soma de `n_animais` da composição.
- `altura(p) = massa[p] ÷ densidade`.
- `proximo_dia_de_manejo(d)`: primeiro dia depois de `d`, dentro de 7 dias, cujo `weekday()` está nos dias preferenciais (pode cair fora do horizonte do plano; tudo bem, a taxa tem 14 dias).

Passo 1: alertas e pedidos do início (`data = data_base`), por piquete em ordem de nome:

- `faltantes` não vazio:

- se algum item for de calibração ou altura (`densidade_kg_ha_por_cm`, `rue_max_g_por_mj`, `eficiencia_pastejo`, `altura_entrada_cm`, `altura_saida_cm`, `altura_maxima_cm`, `altura_minima_cm`) → `AGUARDANDO_PARAMETRO`;
- senão → `ESTIMATIVA_INDISPONIVEL`;
- texto: "0 {nome} ({cultivar_nome}) ainda não recebe recomendação: {motivo_confianca}.", confiança e motivo do próprio piquete.
- Senão, contínuo com estimativa:
 - `altura_hoje > altura_maxima` → `CONTINUO_ACIMA_MAXIMA`: "0 {nome} (pastejo contínuo) está em {h} cm, acima da altura máxima do {cultivar} ({max} cm). Considere colocar mais animais nele.";
 - `altura_hoje < altura_minima` → `CONTINUO_ABAIXO_MINIMA`: "... abaixo da altura mínima do {cultivar} ({min} cm). Considere tirar animais dele."
- **Pedido de validação** (medir com régua) se: o piquete tem estimativa e confiança baixa, ou "`altura_inicial`" está em `faltantes`. `motivo = motivo_confianca` do piquete.
- Lote sem piquete (`piquete_atual_id` nulo) → `LOTE_SEM_PIQUETE`: "0 lote {nome} não está em nenhum piquete. Informe onde ele está na página Lotes.", com a confiança do peso do lote.
- **Antes do primeiro dia de manejo:** se `data_base` **não** é dia de manejo, para cada lote num piquete planejável, simule os dias até o primeiro dia de manejo. Se a altura prevista ficar `<= altura_saida`, gere o alerta `SEM_DIA_DE_MANEJO` (`data = data_base`): "0 lote {lote} deve ficar abaixo de {saida} cm no {piquete} antes do próximo dia de manejo ({ddd} {dd/mm}). Se puder, mova antes." Isso acontece quando o plano é recalculado no meio da semana, e o sistema não pode ficar calado.

Passo 2: simular a semana, para `i` de 0 a 6 (`d = data_base + i`):

1. Se `d` é dia de manejo:

- `n = (proximo_dia_de_manejo(d) - d).days`.
- **Candidatos a sair:** cada lote cujo piquete atual é planejável. Simule `n` dias do piquete com o lote dentro (a partir de `massa[p]`, taxas de índice `i ... i+n-1`, usando `avancar_massa_um_dia`) → `altura_prevista`. Se `altura_prevista <= altura_saida`, o lote é candidato com urgencia(`altura_prevista`, `parametros`).
- Ordene os candidatos por urgência **decrecente**; empate → nome do lote.
- Para cada candidato, em ordem:
 - **Mão de obra:** se já foram movidos animais hoje e `movidos_hoje + n_animais(lote) > capacidade_animais` → alerta `CAPACIDADE_EXCEDIDA`: "Não há mão de obra para mover o lote {lote} ({n} animais) {na/no} {dia por extenso} ({dd/mm}): o limite é {cap} animais por dia. Ele continua no {origem}.". Pule para o próximo. (Se for o **primeiro** lote do dia, ele pode ser movido mesmo sendo maior que a capacidade: lote não se divide.)
 - **Escolher o destino:** entre os piquetes planejáveis, vazios, diferentes da origem e com `descanso_cumprido(descanso[q], descanso_min_dias)`:

- **aptos:** `altura(q) >= altura_entrada`. Escolha o **mais passado do ponto:** o menor ($-(\text{altura}(q) - \text{entrada})$, distância da origem, nome). Assim, o capim que está passando do ponto é pastejado primeiro (ADR-025);
- se não houver apto, **marginais:** `altura(q) >= 0,9 × entrada`. Escolha o menor ($\text{entrada} - \text{altura}(q)$, distância da origem, nome);
- se não houver nenhum → alerta SEM_PIQUETE_APT0: "Nenhum piquete estará pronto para o lote {lote} {na/no} {dia} ({dd/mm}). Ele continua no {origem}, que deve ficar abaixo de {saida} cm antes de {próximo dia de manejo por extenso} ({dd/mm})." O lote fica onde está.

▪ Registrar a movimentação:

- `id = uuid5(plano_id, f"{lote_id}:{d.isoformat()}")`;
- `altura_destino_cm` e `altura_origem_cm` = altura no início do dia `d`, com 1 casa;
- alvos de entrada (destino) e saída (origem);
- `motivo` = parte 1 + " " + parte 2;
- `confianca, motivo_confianca = confianca_movimentacao(destino, lote)`.

Atualize também `movidos_hoje += n_animais(lote)` (zera a cada dia).

- **Passando do ponto** (anote, alerta sai no passo 4): no início de cada dia de manejo, antes das movimentações, todo piquete planejável **vazio** com `altura(q) >= 1,2 × altura_entrada` é anotado com a primeira data e altura em que isso aconteceu.

Atualize: `ocupante[origem] = None`, `descanso[origem] = 0`, `ocupante[destino] = lote`, `posicao[lote] = destino`.

2. **Avançar um dia:** para cada piquete planejável, `massa[p] = avancar_massa_um_dia(massa[p], taxa[i], consumo do ocupante (0 se vazio), area, eficiencia)`. Depois, `descanso[p] += 1` para todo piquete **vazio**.

Passo 3: dias previstos. Para cada movimentação:

- se o mesmo lote tem uma movimentação **posterior** no plano → a diferença em dias;
- senão → `floor(dias_ocupacao(massa do destino no início do dia do movimento, altura_saida × densidade, média das 7 taxas do destino a partir do índice do dia, area, eficiencia, consumo))`. Se der infinito (o capim cresce mais do que o lote come), use 7.

Passo 4: montar o plano.

- **Alerta PASSANDO_DO_PONTO:** para cada piquete anotado que **não** é destino de nenhuma movimentação do plano: "0 {piquete} vai passar do ponto: {h} cm {na/no} {dia} ({dd/mm}), acima de {1,2×entrada} cm (entrada: {entrada} cm). Considere adiantar a entrada de um lote.", com a confiança e o motivo do próprio piquete.
- `movimentacoes` ordenadas por (data, lote_nome); alertas por (data, tipo.value, texto); `pedidos_validacao` por nome do piquete.
- `piquetes`: um `ResumoPiquete` por piquete, na ordem do estado, com `altura_hoje_cm`, alvos de entrada e saída dos `parametros` (nulos se não houver), nome do lote atual, confiança, motivo e faltantes.
- `data_geracao = agora`, `data_inicio = data_base`, `horizonte_dias = 7`.

Textos (português, prontos para o produtor):

- **Parte 1:** "0 {origem} chegaria a {h_prev} cm antes do próximo dia de manejo ({ddd} {dd/mm}), abaixo da saída de {saida} cm."
- **Parte 2 (apto):** "0 {destino} está em {h} cm (ponto de entrada: {entrada} cm)."
- **Parte 2 (marginal):** "Nenhum piquete estará no ponto de entrada; o {destino} é o mais próximo, com {h} cm (alvo: {entrada} cm)."
- **Números:** 1 casa decimal com vírgula, e sem ,0 no fim (31,5, 34, 15).
- **Dia abreviado:** seg ter qua qui sex sáb dom.
- **Dia por extenso:** segunda terça quarta quinta sexta sábado domingo, com “na” antes (e “no” antes de sábado e domingo).

Critérios de aceite:

- [] **Fixture:** com o estado de exemplo, a fazenda do exemplo, o `agora` e o `plano_id` da seção 4, `plano_para_dict(gerar_plano(...))` é **igual** a `plano_exemplo.json`.
- [] **Determinismo:** duas chamadas iguais → planos iguais.
- [] **Dia não preferencial:** nenhuma movimentação fora dos dias preferenciais.
- [] **Lote maior que a capacidade:** lote de 150 animais, capacidade 100, sozinho no dia → é movido.
- [] **Passando do ponto:** piquete vazio a 40 cm (entrada 30) que ninguém recebe → alerta `PASSANDO_DO_PONTO`.
- [] **Mão de obra saturada:** com capacidade de 100 animais e 2 lotes urgentes de 80 no mesmo dia → 1 movimentação (o mais urgente) + 1 `CAPACIDADE_EXCEDIDA`.
- [] **Resíduo:** nenhum lote é mandado para piquete abaixo de 90% da entrada.
- [] **Sem piquete apto:** gera `SEM_PIQUETE_APTO` e o lote fica.
- [] **Contínuo e faltantes:** nunca recebem nem perdem lote; só geram alerta.
- [] **Confiança baixa em tudo:** ainda produz plano, alertas e pedidos, sem exceção.

Fora do escopo: fusão de lotes (F-022); ajuste de lotação no contínuo (F-009B); CP-SAT ou busca local; lookahead de “esperar dois dias”; regra de 1–3 dias de ocupação (ADR-024); qualquer I/O neste arquivo.

J4: Gravar e ler o plano, com status (`persistencia/planos.py`)

Por quê: o plano precisa ficar guardado. O site mostra o plano em vigor, e o bot precisa saber quais movimentações o produtor está respondendo. Existe também o **plano candidato**: quando chegam imagens novas no meio da semana e o plano muda, o produtor escolhe se troca ou não (ADR-025). Por isso cada plano tem um `status`:

status	Significado
vigente	O plano que o produtor está seguindo (só um por fazenda)
candidato	Plano novo oferecido ao produtor, ainda sem resposta (no máximo um)
substituído	Foi vigente e deixou de ser
descartado	Candidato recusado ou vencido por outro

Como:

```

def salvar_plano(conn, plano: PlanoManejo, status: str = "vigente") -> None
def promover_candidato(conn, plano_id: UUID) -> PlanoManejo
def descartar_plano(conn, plano_id: UUID) -> None
def carregar_plano_atual(conn, fazenda_id: UUID) -> PlanoManejo | None      # o vigente
def carregar_candidato(conn, fazenda_id: UUID) -> PlanoManejo | None
def carregar_plano(conn, plano_id: UUID) -> PlanoManejo | None            # qualquer
status
def status_do_plano(conn, plano_id: UUID) -> str | None
def ids_respondidos(conn, fazenda_id: UUID) -> frozenset[UUID]

```

1. `salvar_plano(conn, plano, status)`, COM `status` EM `"vigente"` OU `"candidato"`:
 - SE `"vigente"`: `UPDATE plano SET status = 'substituido' WHERE fazenda_id = %s AND status = 'vigente'`, depois `UPDATE ... SET status = 'descartado' WHERE ... status = 'candidato'`;
 - SE `"candidato"`: SÓ `UPDATE ... SET status = 'descartado' WHERE ... status = 'candidato'`;
 - `INSERT INTO plano (id, fazenda_id, gerado_em, data_inicio, horizonte_dias, payload, status)`, COM `payload = Json(plano_para_dict(plano))` (from `psycpg.types.json` import `Json`);
 - **só quando "vigente"**: para cada movimentação, `registrar_evento(conn, plano.fazenda_id, TipoEvento.MANEJO_RECOMENDADO, OrigemEvento.SISTEMA, ocorrido_em=datetime.now(UTC), payload=..., ator="planner", chave_idempotencia=f"recomendacao:{plano.id}:{mov.id}")` (payload na seção “Entradas e saídas”). Candidato não gera evento: ele ainda não foi recomendado ao produtor.
2. `promover_candidato(conn, plano_id)`: confere que o status é `candidato` (senão `ValueError`), marca o vigente atual como `substituido`, marca este como `vigente`, registra os eventos `manejo_recomendado` dele (como no item 1) e devolve o plano.
3. `descartar_plano(conn, plano_id)`: `UPDATE ... SET status = 'descartado' WHERE id = %s AND status = 'candidato'`.
4. `carregar_plano_atual`: o mais recente com `status = 'vigente'`. `carregar_candidato`: o mais recente com `status = 'candidato'`. `carregar_plano`: por id, qualquer status. `status_do_plano`: o status OU `None`.
5. `ids_respondidos(conn, fazenda_id)`: OS `entidade_id` de todos os eventos `manejo_confirmado`, `manejo_recusado` e `manejo_divergente` da fazenda, ou seja, as movimentações que o produtor já respondeu.
6. **Nenhuma função faz commit**: quem chama faz.

Critérios de aceite:

- [] Salvar e carregar devolve um plano igual ao original.
- [] Depois de salvar um vigente novo, só existe um `vigente` por fazenda.
- [] Candidato não gera evento; `promover_candidato` gera.
- [] Testes com banco pulam se `SEUGADO_TEST_DATABASE_URL` não existir.

Fora do escopo: apagar plano; histórico paginado.

J4b: Comparar o plano em vigor com um plano novo (`planner/comparacao.py`)

Por quê: o sistema recalcula todo dia com as imagens novas do satélite, mas o produtor pode já ter se organizado com o plano que recebeu. Só vale incomodar se o plano novo muda algo que ele **ainda não respondeu**. Esta função diz exatamente o que mudou, lote por lote, e o Leo mostra isso no bot.

Como:

```
def comparar_planos(anterior: PlanoManejo, novo: PlanoManejo,
                    respondidas: frozenset[UUID], hoje: date) -> tuple[DiferencaLote, ...]
```

1. Para cada lote que aparece em qualquer um dos dois planos:
 - `antes` = `PassoPlano(mov.data, mov.piquete_destino_nome)` das movimentações do **anterior** daquele lote com `mov.data >= hoje` e `mov.id` **não** em `respondidas`, em ordem de data;
 - `depois` = o mesmo a partir do **novo** (sem filtro de respondidas).
2. Se `antes != depois`, entra um `DiferencaLote(lote_id, lote_nome, antes, depois)`.
3. Devolva em ordem de `lote_nome`. Vazio = nada relevante mudou.

`PassoPlano` e `DiferencaLote` já existem em `seugado.contratos`.

Critérios de aceite:

- [] Planos iguais → vazio.
- [] Mudança só numa movimentação já respondida → vazio.
- [] Destino diferente na quinta para o lote Recria → um `DiferencaLote` com `antes` e `depois`.
- [] Função pura, sem banco.

J5: Testes e PR

1. `tests/planner/test_otimizador.py` com todos os critérios de J3. Monte estados pequenos direto em Python (2 a 4 piquetes) para os casos isolados, além do teste da fixture.
2. `tests/planner/test_confianca.py` e `tests/persistencia/test_planos.py`.
3. `uv run ruff check .`, `uv run mypy`, `uv run --env-file .env pytest`.
4. PR `feat/joao-otimizador` → `main`, com a saída do plano de exemplo no texto do PR.

6. Quem usa o seu código

- **Leandro**, na rotina (`jobs/ciclo.py`):
 - entrega semanal e recálculo: `gerar_plano` → `salvar_plano(conn, plano, "vigente")`;
 - rotina diária: `gerar_plano` → `comparar_planos(vigente, novo, ids_respondidos(...), hoje)` → se houver diferença, `salvar_plano(conn, novo, "candidato")`;
 - página web: `carregar_plano_atual`.
- **Leo**:
 - `carregar_plano_atual` para os botões “Fiz / Não fiz / Fiz diferente”;
 - `carregar_candidato`, `comparar_planos`, `promover_candidato` e `descartar_plano` para a pergunta “quer ver as mudanças?”;
 - `ids_respondidos` para não perguntar duas vezes.
- **Ezequiel** pinta o mapa com `plano.piquetes` e desenha as setas com `plano.movimentacoes`.

Por isso a saída precisa bater **exatamente** com o contrato: todo mundo desenvolve em cima da fixture.

7. Usando IA no seu fluxo

Dê à IA este documento, `src/seugado/contratos.py` e as duas fixtures, e peça uma tarefa por vez. Exemplo:

“Leia docs/equipe/JOAO.md (seções 4 e J3), `src/seugado/contratos.py` e `tests/fixtures/*.json`. Implemente `src/seugado/planner/otimizador.py` seguindo o algoritmo passo a passo. O teste principal é reproduzir `plano_exemplo.json` exatamente. Não altere nenhum outro arquivo e não adicione dependências.”

Se o seu resultado divergir da fixture, compare movimento a movimento. As diferenças mais comuns: esquecer de contar descanso só para piquete **vazio**, usar a altura do fim do dia em vez do início, e formatar `34,0` em vez de `34`.

8. Glossário de conferência (números da fixture)

- **Segunda 28/09:**

- o Recria (150 novilhos, 1.113,75 kg MS/dia) está no Piquete 2 (22 cm, 3,5 ha); até quinta cairia para 11,6 cm, abaixo da saída de 15 cm, e é o mais urgente;
- as Vacas com bezerro (140 animais) estão no Piquete 5 e cairiam para 11,7 cm;
- os aptos são o Piquete 6 (34 cm) e o 1 (31,5 cm). O **mais passado do ponto** é o 6, que fica com o Recria; as Vacas vão para o 1;
- $150 + 140 = 290$ animais, dentro da capacidade de $2 \times 150 = 300$.

- **Quinta 01/10:**

- o Recria no Piquete 6 chegaria à segunda seguinte com 15,9 cm, então fica;
- as Vacas no 1 chegariam a 7,5 cm, então saem;
- nenhum piquete está a 30 cm, mas o 3 está a 28,6 cm (≥ 27 , que é 90% de 30) e fica com elas.

- **Dias previstos:**

- Recria no 6: 7 (sem nova movimentação na semana; `dias_ocupacao` $\approx 7,4$);
- Vacas no 1: 3 (até quinta);
- Vacas no 3: 4.

- **Alertas:** Piquete 8 (Mombaça, sem calibração) e Piquete 7 (contínuo acima de 35 cm).

- **Pedido de medição:** Piquete 4 (última imagem há 18 dias).